

1. Πόσα Datacenters δημιουργούνται; και πόσοι Hosts δημιουργούνται σε κάθε Datacenter;

1. Δημιουργούνται 2 Datacenters (#2, #3) και 2 Hosts (#0, #1) σε κάθε Datacenter

2. Πόσους επεξεργαστές (PEs) και τι άλλα χαρακτηριστικά έχει κάθε Host (mips, μνήμη, κλπ);

2. Το πρώτο host (host0) έχει 4 cpu cores (PEs) ενώ το host1 2 cores. Καθ'ένα από αυτά έχει μνήμη 2048 MB RAM, αποθηκευτικό χώρο (storage) 1000000 MB ~1TB, Network Bandwidth 10000 Mbps~10 Gbps BW και κάθε PE του υποστηρίζει 1000 MIPS (processing power).

3. Τι άλλα χαρακτηριστικά έχει το κάθε Datacenter; και σε ποια κλάση αναπαρίστανται;

3. Για τα συγκεκριμένα Datacenters είναι τα εξής:

arch= CPU architecture (x86), **os**=operating system (Linux), **vmm** = hypervisor "Xen", **hostList**= all physical machines (hosts) που ανήκουν σε κάθε datacenter, **time_zone** = 10.0 (time zone this resource located), **cost** = 3.0 (the cost of using processing in this resource), **costPerMem** = 0.05 (the cost of using memory in this resource), **costPerStorage** = 0.1 (the cost of using storage in this resource), **costPerBw** = 0.1 (the cost of using bw in this resource). Αναπαρίστανται στην κλάση **DatacenterCharacteristics**. Επίσης στην κλάση **Datacenter** αναπαρίστανται και τα: name, characteristics, vmAllocationPolicy : VmAllocationPolicySimple(hostList), storageList.

VmAllocationPolicySimple: This policy decides **which host gets which VM**. Simple policy = first host with enough resources.

4. Πόσες VM δημιουργούνται; και τι χαρακτηριστικά έχει η κάθε μία;

Σύμφωνα με τις μεταβλητές της μεθόδου **createVM** (η οποία δημιουργεί μια λίστα από VMs) θα δημιουργηθούν συνολικά 20 VMs. Τα χαρακτηριστικά της κάθε μιας είναι:

size = 10000: VM image size in MB (The size of the VM's virtual disk)

ram = 512: This is the amount of RAM memory assigned to the VM, measured in MB.

mips = 1000: This is the processing power of the VM per core, measured in MIPS (Million Instructions Per Second).

bw = 1000: This is the network bandwidth allocated to the VM, measured in Mbps

pesNumber = 1: The number of processing elements (PEs) assigned to the VM. 1 PE = one CPU core.

vmm = "Xen": VMM name (the name of the Virtual Machine Monitor (hypervisor)).

5. Από ποιες παραμέτρους καθορίζεται το πόσες VM μπορεί να φιλοξενήσει κάθε Host; Και πως γίνεται (με τι αλγόριθμο) η προσπάθεια ανάθεσής τους στους διαθέσιμους Hosts (πως αποφασίζεται δηλαδή αν και σε ποιον Host θα τρέξει η κάθε VM);

Ο αριθμός των VMs που μπορεί ένα host να φιλοξενήσει εξαρτάται από τις απαιτήσεις μιας VM όπως: CPU cores (PEs), RAM, Bandwidth, Storage. Ένα host μπορεί να δεχθεί μια VM εάν όλοι οι απαιτούμενοι πόροι είναι διαθέσιμοι. Στη περίπτωση μας για το host0 το όριο (συμφόρησης) είναι ο συνολικός αριθμός των CPU cores (PEs) που είναι 4 και η μνήμη RAM που είναι 2048 MB. Σε αυτή την περίπτωση ο μέγιστος αριθμός VMs που θα φιλοξενηθεί είναι 4. Όσον αφορά το host1 το μοναδικό και μικρότερο όριο είναι ο συνολικός αριθμός των PEs που είναι 2. Σε αυτή την περίπτωση ο μέγιστος αριθμός VMs που θα φιλοξενηθεί είναι 2 (μιας και η RAM μπορεί να φιλοξενήσει έως 4 VMs).

Ο αλγόριθμος που αποφασίζει ποιες VMs θα τρέξουν σε ποιον host ικανοποιώντας όλες τους τις απαιτήσεις είναι η *VM Allocation Policy* και συγκεκριμένα το παράδειγμά μας ο **First-Fit Algorithm** (*VmAllocationPolicySimple*). Αυτός τσεκάρει ένα προς ένα τα hosts με τη σειρά ως προς τις απαιτήσεις των VMs ξεκινώντας με το *Simple policy* = first host with enough resources.

6. Πόσες VMs θα τρέξουν τελικά και σε ποιον Host η κάθε μία; Για ποιο λόγο κάποιες VM δεν θα ανατεθούν σε κανέναν Host; Πώς θα άλλαζαν τα παραπάνω (και γιατί) αν η κάθε VM δημιουργείτο αρχικά με 2 PEs των 250 MIPS και 256 MB RAM;

Στον host0 θα φιλοξενηθούν 4 VMs και στον host1 2 VMs. Συνολικά 6 VMs ανά Datacenter. Κάποιες VM's δεν θα ανατεθούν λόγω του bottleneck στα cores (π.χ. host1) αλλά και στην RAM. Εάν η κάθε VM απαιτούσε 2 PE κατά την τοποθέτηση της τότε ο μέγιστος αριθμός των VMs που θα φιλοξενούσε το host0 θα ήταν 2 και του host1 1 VM μιας και το όριο σε αυτήν την περίπτωση είναι τα 4 PE του host0 και τα 2 PE του host1. Έτσι συνολικά το datacenter θα φιλοξενούσε 3 VMs.

7. Πόσα Cloudlets δημιουργούνται; και με τι χαρακτηριστικά το κάθε ένα;

Σύμφωνα με τον κώδικα δημιουργούνται 40 cloudlets.

Τα χαρακτηριστικά τους είναι:

length = 1000 (the cloudlet requires 1000 million instructions to complete)

fileSize = 300 (cloudlet needs 300 bytes of input data)

outputSize = 300 (the cloudlet produces 300 bytes of output)

```
pesNumber = 1 (the number of CPU cores the cloudlet requires)
UtilizationModelFull
(UtilizationModel utilizationModel = new UtilizationModelFull();)
```

The cloudlet uses 100% of the allocated CPU.

The cloudlet uses 100% of the allocated RAM.

The cloudlet uses 100% of the allocated bandwidth.

8. Πως αποφασίζεται για κάθε Cloudlet σε ποια VM θα τρέξει (με τι αλγόριθμο γίνεται δηλαδή η κατανομή των Cloudlets στις διαθέσιμες VMs); Πόσα Cloudlets τελικά θα εκτελεστούν (τόσο συνολικά σε όλες τις VM όσο και σε κάθε VM ξεχωριστά);

Στο παράδειγμα μας στον CloudSim 4.0 τα Cloudlets ανατίθενται στα VMs με τη σειρά που εμφανίζονται στην VM list δηλαδή σύμφωνα με τον αλγόριθμο *round-robin*.

Θα εκτελεστούν όλα τα 40 cloudlets σε 12 VMs. Σε όλες τις VMs θα ανατεθούν από 3 cloudlets πλην 4 VMs, των #0, #1, #2, #3, στις οποίες θα ανατεθούν από 4 cloudlets.

9. Με τι πολιτική (*space sharing* ή *time sharing* – και σε ποια σημεία του κώδικα φαίνεται / προσδιορίζεται αυτό) γίνεται (α) η δρομολόγηση των Cloudlets στα VMs στα οποία τελικά ανατέθηκαν να τρέχουν; και (β) η δρομολόγηση των VMs στα PEs (ή αλλιώς, η ανάθεση των PEs στα VMs) του Host στον οποίον τρέχουν;

α. Γίνεται με *time sharing* (cloudlets μοιράζονται τις VM CPUs στο χρόνο) και φαίνεται από τον κώδικα *new CloudletSchedulerTimeShared()* (μέσα στο VM creation κώδικα).

β. Γίνεται με *time sharing* (VMs μοιράζονται τα Host CPUs στο χρόνο) και φαίνεται από τον κώδικα (αντικείμενο) *new VmSchedulerTimeShared(peList1)*.

10. Εξηγείστε αναλυτικά το τελικό μέρος των αποτελεσμάτων (πίνακα *outrput* στο τέλος) του παραδείγματος - τι δίνει η κάθε στήλη κλπ, και ειδικότερα μεταξύ των άλλων τους χρόνους εκκίνησης και περάτωσης των Cloudlets (σε άμεση συσχέτιση με τις πολιτικές δρομολόγησης που παρατηρήσατε ότι ακολουθούνται στο ερώτημα 9).

Το τελικό output μας δίνει τις εξής στήλες:

Cloudlet ID	STATUS	Data center ID	VM ID	Time	Start Time	Finish Time
-------------	--------	----------------	-------	------	------------	-------------

Στην *Cloudlet ID* παρατίθεται το ID (από 0 έως 39) κάθε cloudlet, το *STATUS* αυτού (SUCCESS), το *Data center ID* στο οποίο θα ανατεθεί (2 ή 3), η *VM ID* στην οποία θα δρομολογηθεί (από 0 έως 11), ο συνολικός χρόνος *Time* που απαιτείται για να εκτελεστούν όλα τα cloudlets σε κάθε μία από τις διαθέσιμες VMs στις οποίες έχουν ανατεθεί καθώς και η χρονική στιγμή της εκκίνησης *Start Time* που είναι **κοινή** για όλα καθώς και της λήξης *Finish Time* αυτών.

Αυτό που παρατηρούμε από τα αποτελέσματα είναι ότι ο συνολικός χρόνος είναι 3 για τις VMs που θα εξυπηρετήσουν 3 cloudlets και 4 για τις VMs που θα εξυπηρετήσουν 4 cloudlets που είναι οι VMs #0, #1, #2, #3. Τέλος, το έργο (όλα τα cloudlets) θα έχει εκτελεστεί/ολοκληρωθεί σε χρόνο 4.2.

===== OUTPUT =====

Cloudlet ID	STATUS	Data center ID	VM ID	Time	Start Time	Finish Time
4	SUCCESS	2	4	3	0.2	3.2
16	SUCCESS	2	4	3	0.2	3.2
28	SUCCESS	2	4	3	0.2	3.2
5	SUCCESS	2	5	3	0.2	3.2
17	SUCCESS	2	5	3	0.2	3.2
29	SUCCESS	2	5	3	0.2	3.2
6	SUCCESS	3	6	3	0.2	3.2
18	SUCCESS	3	6	3	0.2	3.2
30	SUCCESS	3	6	3	0.2	3.2
7	SUCCESS	3	7	3	0.2	3.2
19	SUCCESS	3	7	3	0.2	3.2
31	SUCCESS	3	7	3	0.2	3.2
8	SUCCESS	3	8	3	0.2	3.2
20	SUCCESS	3	8	3	0.2	3.2
32	SUCCESS	3	8	3	0.2	3.2
10	SUCCESS	3	10	3	0.2	3.2
22	SUCCESS	3	10	3	0.2	3.2
34	SUCCESS	3	10	3	0.2	3.2
9	SUCCESS	3	9	3	0.2	3.2
21	SUCCESS	3	9	3	0.2	3.2
33	SUCCESS	3	9	3	0.2	3.2
11	SUCCESS	3	11	3	0.2	3.2
23	SUCCESS	3	11	3	0.2	3.2
35	SUCCESS	3	11	3	0.2	3.2
0	SUCCESS	2	0	4	0.2	4.2
12	SUCCESS	2	0	4	0.2	4.2
24	SUCCESS	2	0	4	0.2	4.2
36	SUCCESS	2	0	4	0.2	4.2
1	SUCCESS	2	1	4	0.2	4.2
13	SUCCESS	2	1	4	0.2	4.2
25	SUCCESS	2	1	4	0.2	4.2
37	SUCCESS	2	1	4	0.2	4.2
2	SUCCESS	2	2	4	0.2	4.2
14	SUCCESS	2	2	4	0.2	4.2
26	SUCCESS	2	2	4	0.2	4.2
38	SUCCESS	2	2	4	0.2	4.2
3	SUCCESS	2	3	4	0.2	4.2
15	SUCCESS	2	3	4	0.2	4.2

27	SUCCESS	2	3	4	0.2	4.2
39	SUCCESS	2	3	4	0.2	4.2

Αφού απαντήσετε στα παραπάνω ερωτήματα, ασχοληθείτε στη συνέχεια με τα παρακάτω:

A. Τρέξτε ξανά το παράδειγμα με τις υπόλοιπες εναλλακτικές δυνατότητες που σας παρέχονται (*space/time sharing*) για τα σημεία 9(α) και 9(β) παραπάνω, δηλαδή (α) για τη δρομολόγηση των Cloudlets στα VMs, και (β) για τη δρομολόγηση των VMs στους Hosts. Μελετήστε πάλι το output σε κάθε περίπτωση, δείτε αν υπάρχουν ή όχι διαφορές, και εξηγήστε το γιατί. Τρέξτε επίσης ξανά το παράδειγμα εφαρμόζοντας για τη δρομολόγηση των VMs πολιτική *time sharing* με *over-subscription* και περιγράψτε-εξηγήστε πάλι το output.

α)

- CloudletSchedulerTimeShared → CloudletSchedulerSpaceShared (με VmSchedulerTimeShared)

Αλλάζοντας απλά το scheduling μόνο στα cloudlets δεν προκύπτει κάποια αλλαγή σε σχέση με την κατανομή (round-robin) στα υπάρχοντα VMs, το VM scheduling παραμένει το ίδιο μιας και ο CloudletScheduler επηρεάζει μόνο πως τα cloudlets τρέχουν μέσα σ' ένα VM και όχι που τρέχουν. Αυτό που αλλάζει είναι η συμπεριφορά στην εκτέλεση των cloudlets ανά VM. Λόγω του *Space Sharing* δεν έχουμε μοίρασμα των VMs MIPS χρονικά αλλά αποκλειστικό τρέξιμο/ανάθεση ενός cloudlet πάνω σε μία VM τη φορά. Τα υπόλοιπα θα περιμένουν στην ουρά μέχρι το προηγούμενο να έχει τελειώσει.

Έτσι, παρατηρώντας το output του κώδικα βλέπουμε καταρχάς τα πρώτα cloudlets (αφού ανατεθούν στις διαθέσιμες VMs) να ξεκινούν σε χρόνο 0.2 και να τελειώνουν σε 1.2 (συνολικός χρόνος εκτέλεσης ανά cloudlet 1 όπως και στο προηγούμενο output). Ενώ τα επόμενα ανά αντίστοιχη VM να ξεκινούν από χρόνο 1.2, αφού τελειώσουν κι αυτά από 2.2 κι ούτω καθεξής έως ότου εκτελεστούν όλα σε χρόνο 4.2 (όπως και στο προηγούμενο παράδειγμα).

===== OUTPUT =====

Cloudlet ID	STATUS	Data center ID	VM ID	Time	Start Time	Finish Time
0	SUCCESS	2	0	1	0.2	1.2

1	SUCCESS	2	1	1	0.2	1.2
2	SUCCESS	2	2	1	0.2	1.2
4	SUCCESS	2	4	1	0.2	1.2
3	SUCCESS	2	3	1	0.2	1.2
5	SUCCESS	2	5	1	0.2	1.2
6	SUCCESS	3	6	1	0.2	1.2
7	SUCCESS	3	7	1	0.2	1.2
8	SUCCESS	3	8	1	0.2	1.2
10	SUCCESS	3	10	1	0.2	1.2
9	SUCCESS	3	9	1	0.2	1.2
11	SUCCESS	3	11	1	0.2	1.2
12	SUCCESS	2	0	1	1.2	2.2
13	SUCCESS	2	1	1	1.2	2.2
14	SUCCESS	2	2	1	1.2	2.2
16	SUCCESS	2	4	1	1.2	2.2
15	SUCCESS	2	3	1	1.2	2.2
17	SUCCESS	2	5	1	1.2	2.2
18	SUCCESS	3	6	1	1.2	2.2
19	SUCCESS	3	7	1	1.2	2.2
20	SUCCESS	3	8	1	1.2	2.2
22	SUCCESS	3	10	1	1.2	2.2
21	SUCCESS	3	9	1	1.2	2.2
23	SUCCESS	3	11	1	1.2	2.2
24	SUCCESS	2	0	1	2.2	3.2
25	SUCCESS	2	1	1	2.2	3.2
26	SUCCESS	2	2	1	2.2	3.2

28	SUCCESS	2	4	1	2.2	3.2
27	SUCCESS	2	3	1	2.2	3.2
29	SUCCESS	2	5	1	2.2	3.2
30	SUCCESS	3	6	1	2.2	3.2
31	SUCCESS	3	7	1	2.2	3.2
32	SUCCESS	3	8	1	2.2	3.2
34	SUCCESS	3	10	1	2.2	3.2
33	SUCCESS	3	9	1	2.2	3.2
35	SUCCESS	3	11	1	2.2	3.2
36	SUCCESS	2	0	1	3.2	4.2
37	SUCCESS	2	1	1	3.2	4.2
38	SUCCESS	2	2	1	3.2	4.2
39	SUCCESS	2	3	1	3.2	4.2

β)

- **VmSchedulerTimeShared** → **VmSchedulerSpaceShared** (με *CloudletSchedulerTimeShared*)

Παίρνουμε τα ίδια αποτελέσματα όπως στο σενάριο 10. Ο λόγος είναι ο εξής:

Ο VmSchedulerSpaceShared scheduling αλγόριθμος σημαίνει ότι σε κάθε VM ανατίθενται αποκλειστικές PEs. Δεν μπορούν π.χ. 2 VMs να μοιραστούν χρονικά μία PE. Στο παράδειγμά μας κάθε VM απαιτεί 1 PE. Επίσης το host0 έχει 4 PEs και το host1 2 PEs διαθέσιμες (σύνολο 6 ανά datacenter). Εφόσον έχουν τοποθετηθεί συνολικά 6 VMs σε κάθε datacenter κανένα host δεν θα υπολείπεται των PEs, κανένα VM δεν θα απορρίπτεται και κανένα δεν θα περιμένει αναγκαστικά στην ουρά για να εκτελέσει. Άρα εδώ ο SpaceShared αλγόριθμος δεν θα έχει την ευκαιρία να συμπεριφερθεί διαφορετικά.

Τέλος, η εκτέλεση των cloudlets θα γίνει Timeshared όπως και στο σενάριο 10.

- **VmSchedulerTimeShared** → **VmSchedulerSpaceShared** (με *CloudletSchedulerSpaceShared*)

Παίρνουμε ακριβώς τα ίδια αποτελέσματα με το output του Α.α) τα οποία εξαρτώνται αποκλειστικά από τον αλγόριθμο του CloudletSchedulerSpaceShared.

γ)

- time sharing με over-subscription

Τέλος αλλάζοντας την πολιτική time sharing με **over-subscription** αντικαθιστώντας την παράμετρο *new VmSchedulerTimeShared(peList)* με την *new VmSchedulerTimeSharedOverSubscription(peList)* έχουμε σαν αποτέλεσμα την ανάθεση περισσότερων VMs σε ένα PE από τον αριθμό των PEs που είναι διαθέσιμα κατά το χρόνο τοποθέτησης τους. Συγκεκριμένα, ενώ στα προηγούμενα παραδείγματα στο **Host1** ανατίθονταν 2 VMs όσοι κι ο αριθμός των PEs τώρα θα ανατεθούν 4 VMs (οι διπλάσιες). Ο περιοριστικός παράγοντας θα είναι η μνήμη RAM και για τα 2 Hosts! Σύμφωνα με το output θα είναι συνολικά για το Datacenter#2 Host1 οι VM3, VM5, VM6, VM7 και για το Datacenter#3 Host1 οι VM11, VM13, VM14, VM15. Αυτό θα έχει σαν αποτέλεσμα ο συνολικός χρόνος εκτέλεσης των cloudlets που θα ανατεθούν στις συγκεκριμένες VMs να διπλασιαστεί. Δηλαδή, αυτές που στο παράδειγμα 10 (πολιτική *TimeShared*) στο Host1 χρειάζονταν συνολικό χρόνο εκτέλεσης 3 τώρα θα χρειαστούν 6 ενώ αυτές που χρειάζονται τώρα 2 θα εκτελεστούν σε 4. Συνολικός χρόνος εκτέλεσης του 'έργου' θα είναι 6 (sec). Συνολικά και στα δυο Datacenters κάποιες θα εκτελέσουν τα cloudlets τους σε 2, κάποιες σε 3, σε 4, σε 6. Ο χρόνος εκκίνησης όλων θα είναι κοινός (time shared).

===== OUTPUT =====

Cloudlet ID	STATUS	Data center ID	VM ID	Time	Start Time	Finish Time
8	SUCCESS	3	8	2	0.2	2.2
24	SUCCESS	3	8	2	0.2	2.2
9	SUCCESS	3	9	2	0.2	2.2
25	SUCCESS	3	9	2	0.2	2.2
10	SUCCESS	3	10	2	0.2	2.2
26	SUCCESS	3	10	2	0.2	2.2
12	SUCCESS	3	12	2	0.2	2.2
28	SUCCESS	3	12	2	0.2	2.2
0	SUCCESS	2	0	3	0.2	3.2
16	SUCCESS	2	0	3	0.2	3.2
32	SUCCESS	2	0	3	0.2	3.2

1	SUCCESS	2	1	3	0.2	3.2
17	SUCCESS	2	1	3	0.2	3.2
33	SUCCESS	2	1	3	0.2	3.2
2	SUCCESS	2	2	3	0.2	3.2
18	SUCCESS	2	2	3	0.2	3.2
34	SUCCESS	2	2	3	0.2	3.2
4	SUCCESS	2	4	3	0.2	3.2
20	SUCCESS	2	4	3	0.2	3.2
36	SUCCESS	2	4	3	0.2	3.2
11	SUCCESS	3	11	4	0.2	4.2
27	SUCCESS	3	11	4	0.2	4.2
13	SUCCESS	3	13	4	0.2	4.2
29	SUCCESS	3	13	4	0.2	4.2
14	SUCCESS	3	14	4	0.2	4.2
30	SUCCESS	3	14	4	0.2	4.2
15	SUCCESS	3	15	4	0.2	4.2
31	SUCCESS	3	15	4	0.2	4.2
3	SUCCESS	2	3	6	0.2	6.2
19	SUCCESS	2	3	6	0.2	6.2
35	SUCCESS	2	3	6	0.2	6.2
5	SUCCESS	2	5	6	0.2	6.2
21	SUCCESS	2	5	6	0.2	6.2
37	SUCCESS	2	5	6	0.2	6.2

6	SUCCESS	2	6	6	0.2	6.2
22	SUCCESS	2	6	6	0.2	6.2
38	SUCCESS	2	6	6	0.2	6.2
7	SUCCESS	2	7	6	0.2	6.2
23	SUCCESS	2	7	6	0.2	6.2
39	SUCCESS	2	7	6	0.2	6.2

B. Πραγματοποιήστε τις ελάχιστες απαιτούμενες αλλαγές στη διατιθέμενη υποδομή (hosts, PEs και χαρακτηριστικά τους) ώστε να μπορούν να τρέξουν (χωρίς over-subscription) όλες οι VMs. Δώστε τρεις εκδοχές, (α) μία χωρίς να αυξήσετε τον αριθμό hosts και PEs (αυξάνοντας δηλαδή μόνο τα χαρακτηριστικά τους), (β) μία αυξάνοντας τον αριθμό των PEs (χωρίς να αυξήσετε τον αριθμό των hosts), και (γ) μία αυξάνοντας τον αριθμό των hosts. Μελετήστε πάλι το output, δείτε αν υπάρχουν ή όχι διαφορές, και εξηγήστε το γιατί.

(α) Οι απαιτήσεις της κάθε VM όσον αφορά τις MIPS (million instructions per sec) είναι 1000 ενώ σε κύρια μνήμη RAM είναι 512 MB. Αυξάνοντας τον αριθμό των MIPS από 1000 σε 2000 ανά PE σε κάθε host και data center διπλασιάζουμε τον αριθμό των VMs που μπορούν να ανατεθούν σε κάθε PE (cpu core) εφόσον έχουμε αυξήσει ανάλογα με τις συνολικές απαιτήσεις και την χωρητικότητα στην μνήμη RAM του κάθε μηχανήματος (host). Πιο συγκεκριμένα θα δημιουργηθούν και θα ανατεθούν στην διαθέσιμη υποδομή $2X4+2X2=12$ VMs στο Data center#2 ενώ οι υπόλοιπες 8 VMs στο Data center #3 (σύμφωνα με τον κώδικα θα δημιουργηθούν 20 VMs). Στο Datacenter#2 θα είναι: Host#0: VM0, VM1, VM2, VM4, VM6, VM8, VM10, VM11 και Host#1: VM3, VM5, VM7, VM9. Στο Data center#3 θα είναι: Host#0: VM12, VM13, VM14, VM16, VM18 και Host#1: VM15, VM17, VM19. Σε κάθε μία από αυτές θα τρέξουν από 2 cloudlets. Όλα τελειώνουν σε χρόνο 2.2 (sec) με συνολικό χρόνο εκτέλεσης 2 sec η κάθε μία.

===== OUTPUT =====

Cloudlet ID	STATUS	Data center ID	VM ID	Time	Start Time	Finish Time
0	SUCCESS	2	0	2	0.2	2.2
20	SUCCESS	2	0	2	0.2	2.2
1	SUCCESS	2	1	2	0.2	2.2
21	SUCCESS	2	1	2	0.2	2.2
2	SUCCESS	2	2	2	0.2	2.2
22	SUCCESS	2	2	2	0.2	2.2
4	SUCCESS	2	4	2	0.2	2.2
24	SUCCESS	2	4	2	0.2	2.2
6	SUCCESS	2	6	2	0.2	2.2
26	SUCCESS	2	6	2	0.2	2.2
8	SUCCESS	2	8	2	0.2	2.2

28	SUCCESS	2	8	2	0.2	2.2
10	SUCCESS	2	10	2	0.2	2.2
30	SUCCESS	2	10	2	0.2	2.2
11	SUCCESS	2	11	2	0.2	2.2
31	SUCCESS	2	11	2	0.2	2.2
3	SUCCESS	2	3	2	0.2	2.2
23	SUCCESS	2	3	2	0.2	2.2
5	SUCCESS	2	5	2	0.2	2.2
25	SUCCESS	2	5	2	0.2	2.2
7	SUCCESS	2	7	2	0.2	2.2
27	SUCCESS	2	7	2	0.2	2.2
9	SUCCESS	2	9	2	0.2	2.2
29	SUCCESS	2	9	2	0.2	2.2
12	SUCCESS	3	12	2	0.2	2.2
32	SUCCESS	3	12	2	0.2	2.2
13	SUCCESS	3	13	2	0.2	2.2
33	SUCCESS	3	13	2	0.2	2.2
14	SUCCESS	3	14	2	0.2	2.2
34	SUCCESS	3	14	2	0.2	2.2
16	SUCCESS	3	16	2	0.2	2.2
36	SUCCESS	3	16	2	0.2	2.2
18	SUCCESS	3	18	2	0.2	2.2
38	SUCCESS	3	18	2	0.2	2.2
15	SUCCESS	3	15	2	0.2	2.2
35	SUCCESS	3	15	2	0.2	2.2
17	SUCCESS	3	17	2	0.2	2.2
37	SUCCESS	3	17	2	0.2	2.2
19	SUCCESS	3	19	2	0.2	2.2
39	SUCCESS	3	19	2	0.2	2.2

CloudSimExample6 finished!

(β) Στο συγκεκριμένο ερώτημα αυξάνουμε (με τον κατάλληλο κώδικα) τον αριθμό των cores (PEs) ισάριθμα κατά 2 σε κάθε host. Στο host1 δεν χρειάζεται να κάνουμε άλλη παρέμβαση μιας και η RAM είναι αρκετή για να φιλοξενήσει 2 ακόμα PEs. Αντίθετα, στο host0 αυξάνουμε την μνήμη κατά 1024 MB ώστε να φθάσει στα 3072 MB για να πάρει άλλες 2 PEs. Έτσι, στο output θα δημιουργηθούν συνολικά 20 VMs (δηλαδή 10 σε κάθε Datacenter). Σε κάθε μία από αυτές θα τρέξουν ομοιόμορφα από 2 cloudlets (όπως στο προηγούμενο παράδειγμα) και όλες θα τελειώνουν στον ίδιο χρόνο με συνολικό χρόνο εκτέλεσης 2 sec η κάθε μία.

===== OUTPUT =====

Cloudlet ID	STATUS	Data center ID	VM ID	Time	Start Time	Finish Time
0	SUCCESS	2	0	2	0.2	2.2
20	SUCCESS	2	0	2	0.2	2.2
1	SUCCESS	2	1	2	0.2	2.2
21	SUCCESS	2	1	2	0.2	2.2
2	SUCCESS	2	2	2	0.2	2.2
22	SUCCESS	2	2	2	0.2	2.2
4	SUCCESS	2	4	2	0.2	2.2
24	SUCCESS	2	4	2	0.2	2.2
6	SUCCESS	2	6	2	0.2	2.2
26	SUCCESS	2	6	2	0.2	2.2

8	SUCCESS	2	8	2	0.2	2.2
28	SUCCESS	2	8	2	0.2	2.2
3	SUCCESS	2	3	2	0.2	2.2
23	SUCCESS	2	3	2	0.2	2.2
5	SUCCESS	2	5	2	0.2	2.2
25	SUCCESS	2	5	2	0.2	2.2
7	SUCCESS	2	7	2	0.2	2.2
27	SUCCESS	2	7	2	0.2	2.2
9	SUCCESS	2	9	2	0.2	2.2
29	SUCCESS	2	9	2	0.2	2.2
10	SUCCESS	3	10	2	0.2	2.2
30	SUCCESS	3	10	2	0.2	2.2
11	SUCCESS	3	11	2	0.2	2.2
31	SUCCESS	3	11	2	0.2	2.2
12	SUCCESS	3	12	2	0.2	2.2
32	SUCCESS	3	12	2	0.2	2.2
14	SUCCESS	3	14	2	0.2	2.2
34	SUCCESS	3	14	2	0.2	2.2
16	SUCCESS	3	16	2	0.2	2.2
36	SUCCESS	3	16	2	0.2	2.2
18	SUCCESS	3	18	2	0.2	2.2
38	SUCCESS	3	18	2	0.2	2.2
13	SUCCESS	3	13	2	0.2	2.2
33	SUCCESS	3	13	2	0.2	2.2
15	SUCCESS	3	15	2	0.2	2.2
35	SUCCESS	3	15	2	0.2	2.2
17	SUCCESS	3	17	2	0.2	2.2
37	SUCCESS	3	17	2	0.2	2.2
19	SUCCESS	3	19	2	0.2	2.2
39	SUCCESS	3	19	2	0.2	2.2

(v) Στην άσκηση αυτή προκειμένου να φιλοξενήσουμε και να τρέξουν 20 VMs αυξάνουμε τα hosts κατά ένα (από 2 σε 1) σε κάθε Datacenter και συγκεκριμένα υλοποιούμε χρησιμοποιώντας τον κώδικα της Java άλλο ένα host με τα χαρακτηριστικά του υπάρχοντος host#0, δηλ. με χαρακτηριστικά όπως 4 PEs των 1000 MIPS η κάθε μία και μνήμη RAM 4 GB. Το αποτέλεσμα είναι να ανατεθούν και δημιουργηθούν 4 VMs ακόμα ανά Datacenter έτσι ώστε στο τέλος να φτάσουμε τον αριθμό των 20 VMs που είναι και η απαίτηση μας. Σε κάθε μία από αυτές θα τρέξουν ομοιόμορφα από 2 cloudlets (όπως στο προηγούμενο παράδειγμα) και όλες θα τελειώνουν στον ίδιο χρόνο με συνολικό χρόνο εκτέλεσης 2 sec η κάθε μία.

===== OUTPUT =====

Cloudlet ID	STATUS	Data center ID	VM ID	Time	Start Time	Finish Time
0	SUCCESS	2	0	2	0.2	2.2
20	SUCCESS	2	0	2	0.2	2.2
2	SUCCESS	2	2	2	0.2	2.2
22	SUCCESS	2	2	2	0.2	2.2
4	SUCCESS	2	4	2	0.2	2.2
24	SUCCESS	2	4	2	0.2	2.2
7	SUCCESS	2	7	2	0.2	2.2
27	SUCCESS	2	7	2	0.2	2.2
5	SUCCESS	2	5	2	0.2	2.2
25	SUCCESS	2	5	2	0.2	2.2

8	SUCCESS	2	8	2	0.2	2.2
28	SUCCESS	2	8	2	0.2	2.2
1	SUCCESS	2	1	2	0.2	2.2
21	SUCCESS	2	1	2	0.2	2.2
3	SUCCESS	2	3	2	0.2	2.2
23	SUCCESS	2	3	2	0.2	2.2
6	SUCCESS	2	6	2	0.2	2.2
26	SUCCESS	2	6	2	0.2	2.2
9	SUCCESS	2	9	2	0.2	2.2
29	SUCCESS	2	9	2	0.2	2.2
10	SUCCESS	3	10	2	0.2	2.2
30	SUCCESS	3	10	2	0.2	2.2
12	SUCCESS	3	12	2	0.2	2.2
32	SUCCESS	3	12	2	0.2	2.2
14	SUCCESS	3	14	2	0.2	2.2
34	SUCCESS	3	14	2	0.2	2.2
17	SUCCESS	3	17	2	0.2	2.2
37	SUCCESS	3	17	2	0.2	2.2
15	SUCCESS	3	15	2	0.2	2.2
35	SUCCESS	3	15	2	0.2	2.2
18	SUCCESS	3	18	2	0.2	2.2
38	SUCCESS	3	18	2	0.2	2.2
11	SUCCESS	3	11	2	0.2	2.2
31	SUCCESS	3	11	2	0.2	2.2
13	SUCCESS	3	13	2	0.2	2.2
33	SUCCESS	3	13	2	0.2	2.2
16	SUCCESS	3	16	2	0.2	2.2
36	SUCCESS	3	16	2	0.2	2.2
19	SUCCESS	3	19	2	0.2	2.2
39	SUCCESS	3	19	2	0.2	2.2

C. Σε ποια σημεία του κώδικα (υλοποίησης του *simulator*) και πώς θα επεμβαίνατε (α) για να αλλάξετε την πολιτική / αλγόριθμο απόφασης για τα 5. και 8., και (β) για να προσθέσετε κάποια άλλη δικιά σας πολιτική για τα 9(α) και 9(β); Ειδικότερα όσον αφορά το 5. (την πολιτική που ακολουθείται δηλαδή για την ανάθεση των VMs στους Hosts), εξηγήστε με περισσότερη λεπτομέρεια πώς θα μπορούσατε να ενσωματώσετε μία άλλη πολιτική της επιλογής σας. Αναζητήστε στη συνέχεια και επιλέξτε μία άλλη συγκεκριμένη πολιτική

(συμβουλευτείτε μεταξύ άλλων και τα επισυναπτόμενα links και papers – βλ. folder ‘VM Allocation Policies’), περιγράψτε τη σύντομα, και προσπαθήστε να την υλοποιήσετε.

(α) Στην τωρινή κατάσταση το πρόγραμμα χρησιμοποιεί την `VmAllocationPolicySimple` = **First-Fit** κατανομή. Σκανάρει όλη την `hostlist` με τη σειρά, επιλέγει το πρώτο `host` το οποίο έχει αρκετή ελεύθερη (σύμφωνα με τις απαιτήσεις) μνήμη RAM, BW και PEs και δημιουργεί την πρώτη VM πάνω στο `host`. Αφού τελειώσει με αυτό και δεν υπάρχει κάποιο θέμα όσον αφορά τις απαιτήσεις ώστε να επιλέξει πρόωρα κάποιο άλλο `host` συνεχίζει με το επόμενο διαθέσιμο.

Με τον `ClouSim` μπορούμε να αλλάξουμε την υφιστάμενη πολιτική αντικαθιστώντας την `VmAllocationPolicySimple` με μία ενσωματωμένη (built-in) πολιτική ή ακόμα και μία προσαρμοσμένη custom δικιά μας. Αυτό θα γίνει με το πέρασμα μιας διαφορετικής κλάσης στον `Datacenter constructor` όπου στον κώδικα περιγράφεται όπως: `new Datacenter(name, characteristics, new VmAllocationPolicySimple(hostList), storageList, 0);` αντικαθιστώντας την παράμετρο `VmAllocationPolicySimple` με μια `VmAllocationPolicyCustom`. Η νέα κλάση που θα δημιουργήσουμε θα κάνει `extend` την υπάρχουσα κλάση `VmAllocationPolicySimple` κάνοντας `override` την μέθοδο `allocateHostForVm` με τον νέο αλγόριθμο.

Μερικές τέτοιες custom πολιτικές/αλγόριθμοι που μπορούμε να εφαρμόσουμε είναι: Best-Fit, Worst-Fit, Round-Robin, Least-Loaded, κτλ. Ο αλγόριθμος που θα εφαρμόσουμε είναι ο Least-Loaded. Βάσει αυτής της πολιτικής τοποθετούμε την κάθε VM στο `host` με την μικρότερη CPU χρήση (lowest CPU utilization) κι όχι απλά στο πρώτο ελεύθερο `host`. Καθώς έχουμε 2 διαφορετικά `hosts` όσον αφορά τα PEs, 4 το `host0` και 2 το `host1`, θα αποφύγουμε να υπερφορτώσουμε (overload) από πολύ νωρίς το `host1`. Οι αλλαγές στον κώδικα της άσκησης `CloudSimExample6` και συγκεκριμένα στο `Datacenter` object θα είναι: από `new VmAllocationPolicySimple(hostList)` σε **`new VmAllocationPolicyLeastLoaded(hostList)`**. Ο κώδικας της νέας java κλάσης **`VmAllocationPolicyLeastLoaded`** παρατίθεται εδώ,



`VmAllocationPolicyLeastLoaded.docx`

και θα τοποθετηθεί στο πακέτο `org.cloudbus.cloudsim` και θα εισαχθεί στον κώδικα της άσκησης μας με την `import org.cloudbus.cloudsim.VmAllocationPolicyLeastLoaded;`

Με ανάλογο τρόπο θα μπορούσα να αλλάξω και αλγόριθμο/πολιτική και για την ανάθεση των `cloudlets` στις VMs ή με άλλα λόγια την `Cloudlet Scheduling Policy`. Η πρωταρχική `cloudlet scheduling` πολιτική είναι `time-shared` η οποία συμπεριφέρεται σαν `round-robin`. Εάν θέλω να την αλλάξω αυτό πρέπει να γίνει μέσα από την `createVM()` μέθοδο, όπως π.χ. αντικαθιστώντας το object `new CloudletSchedulerTimeShared()` με την `new CloudletSchedulerSpaceShared()` εάν

θέλουμε να υλοποιήσουμε την SpaceShared πολιτική που εξηγήσαμε πιο πάνω ή με την *new CloudletSchedulerDynamicWorkload(mips, bw)* εάν θέλουμε να υλοποιήσουμε το Dynamic Workload Scheduling. Στο τελευταίο θα πρέπει να γίνει αλλαγή και στο UtilizationModel δηλ. από *new UtilizationModelFull()* σε *new UtilizationModelDynamic*. Οι αντίστοιχες κλάσεις εφόσον είναι ενσωματωμένες στο package *org/cloudbus/cloudsim* θα πρέπει να εισαχθούν με την *import* command στο πρόγραμμά μας.

(β) Ενεργούμε όπως στο ερώτημα Α και για την κατανομή των cloudlets στις VMs και για την κατανομή των VMs στους PEs. Στην πρώτη περίπτωση αντικαθιστώντας μέσα από την *createVM()* μέθοδο το object *new CloudletSchedulerTimeShared()* με την *new CloudletSchedulerSpaceShared()* εάν θέλουμε να υλοποιήσουμε την SpaceShared πολιτική και στην 2^η περίπτωση αντικαθιστώντας στη μέθοδο *createDatacenter()* και συγκεκριμένα στη δημιουργία του *new host()* το object *new VmSchedulerTimeShared(peList1)* με *new VmSchedulerSpaceShared(peList1)* και με *new VmSchedulerSpaceShared(peList2)*. Μια άλλη επιλογή θα ήταν επίσης η opportunistic space-shared allocation policy με *new VmSchedulerOpportunisticSpaceShared(peList1)* και *new VmSchedulerOpportunisticSpaceShared(peList2)*. Όλες οι αντίστοιχες κλάσεις θα πρέπει να εισαχθούν μέσω *import* στο πρόγραμμά μας.